

LAB DAY 5

NAME: Noelshaun Deril.W

REG. NO.: 192324174

COURSE CODE AND NAME: CSA0617 DESIGN AND ANALYSIS OF ALGORITHM

0/1 KNAPSACK PROBLEM (Dynamic Programming)

PYTHON CODE:

```
def knapsack(weights, values, capacity):  
    n = len(weights)  
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]  
    for i in range(1, n + 1):  
        for w in range(capacity + 1):  
            if weights[i - 1] <= w:  
                dp[i][w] = max(  
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],  
                    dp[i - 1][w]  
                )  
            else:  
                dp[i][w] = dp[i - 1][w]  
    return dp[n][capacity]  
  
weights = [1, 3, 4, 5]  
values = [1, 4, 5, 7]  
capacity = 7  
  
print("Maximum Profit:", knapsack(weights, values, capacity))
```

EXECUTION & OUTPUT:

```
AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)
File Edit Format Run Options Window Help
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print("Maximum Profit:", knapsack(weights, values, capacity))
```

TRAVELLING SALES PERSON PROBLEM

PYTHON CODE:

```
from functools import lru_cache
def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    graph[pos][city] + visit(mask | (1 << city), city)
                )
        return ans
    return visit(1, 0)
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost:", tsp(graph))
```

EXECUTION & OUTPUT:

```
*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
from functools import lru_cache
def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans,
                    graph[pos][city] +
                    visit(mask | (1 << city), city)
                )
        return ans
    return visit(1, 0)
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost:", tsp(graph))
|
```

OPTIMAL BINARY SEARCH TYPE (OBST)

PYTHON CODE:

```
def optimal_bst(keys, freq):
```

```
    n = len(keys)
```

```
    cost = [[0 for _ in range(n)] for _ in range(n)]
```

```
    for i in range(n):
```

```
        cost[i][i] = freq[i]    for
```

```
        length in range(2, n + 1):    for i
```

```
            in range(n - length + 1):
```

```
                j = i + length - 1    cost[i][j]
```

```
                = float('inf')    total_freq =
```

```

sum(freq[i:j + 1])      for r in
range(i, j + 1):

    left = cost[i][r - 1] if r > i else 0

right = cost[r + 1][j] if r < j else 0

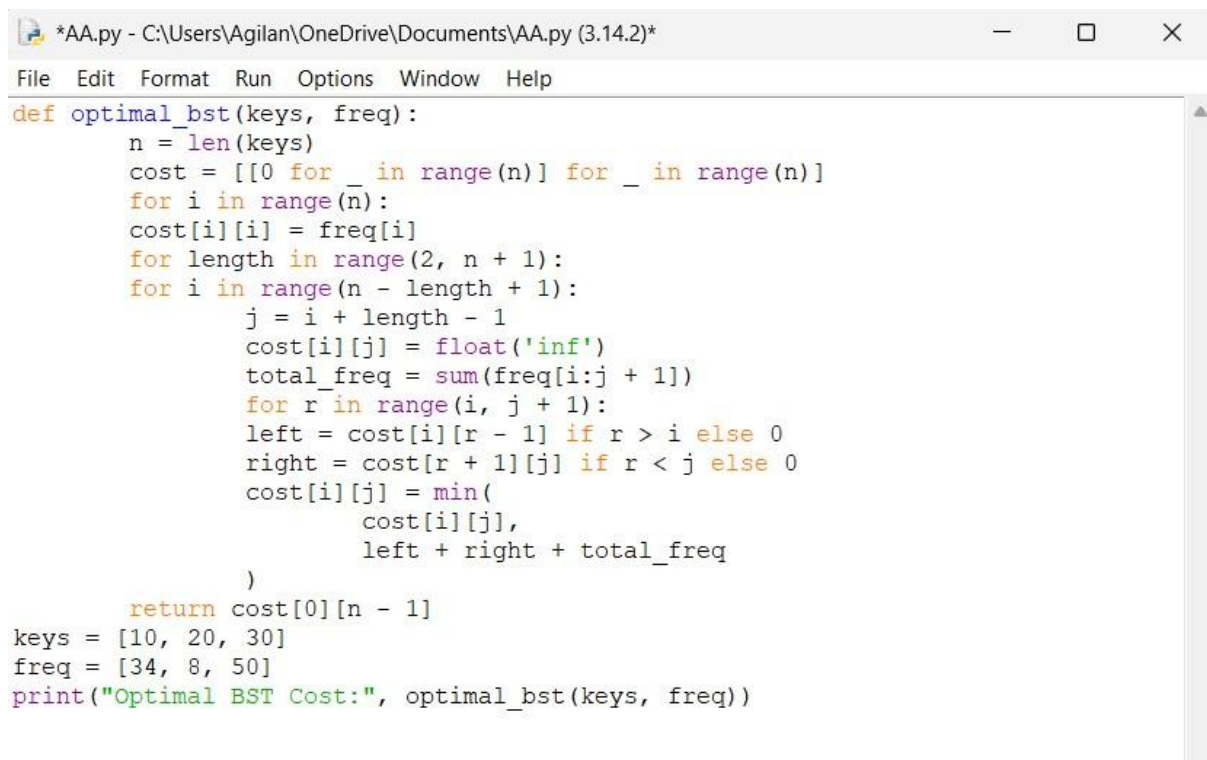
cost[i][j] = min(
    cost[i][j],
    left + right + total_freq
)

return cost[0][n - 1]

keys = [10, 20, 30] freq = [34, 8, 50] print("Optimal
BST Cost:", optimal_bst(keys, freq))

```

EXECUTION & OUTPUT:



The screenshot shows a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The code in the editor is as follows:

```

def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )
    return cost[0][n - 1]

keys = [10, 20, 30]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(keys, freq))

```

0/1 KNAPSACK PROBLEM PROGRAM

PYTHON CODE:

```
def knapsack(wt, val, W):
```

```

n = len(wt)

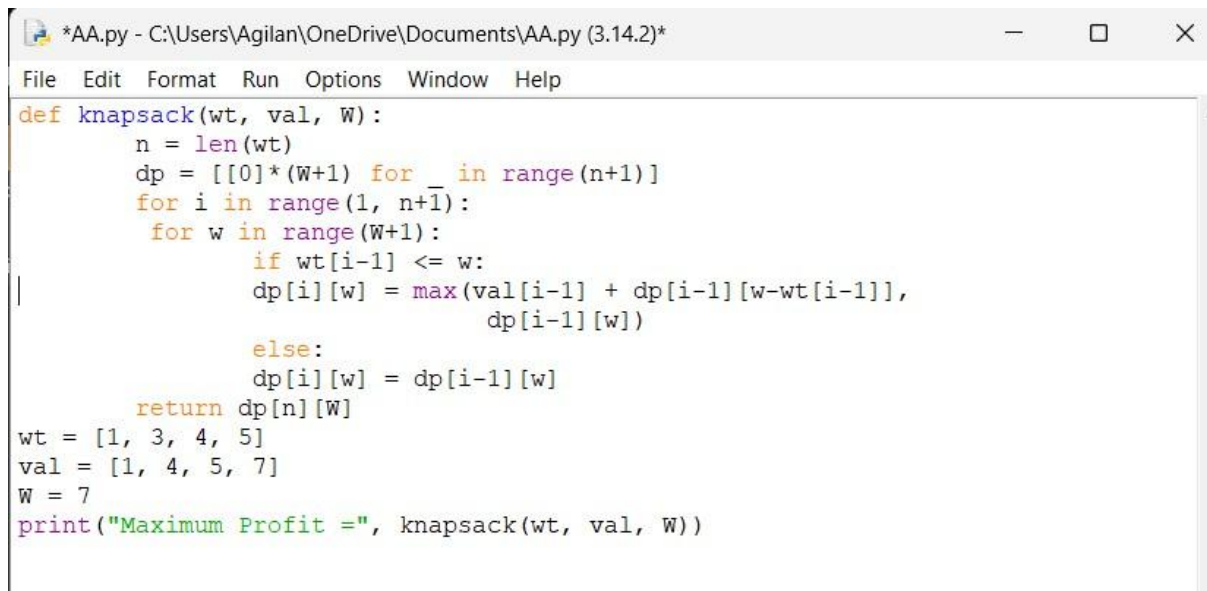
dp = [[0]*(W+1) for _ in
range(n+1)]    for i in range(1, n+1):
for w in range(W+1):
    if wt[i-1] <= w:
        dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
dp[i-1][w])    else:
        dp[i][w] = dp[i-1][w]

    return dp[n][W]

wt = [1, 3, 4, 5] val
= [1, 4, 5, 7]
W = 7 print("Maximum Profit =", knapsack(wt,
val, W))

```

EXECUTION & OUTPUT:



```

*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                    dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
print("Maximum Profit =", knapsack(wt, val, W))

```

0/1 KNAPSACK USING MEMOIZATION PROGRAM

PYTHON CODE:

```

def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:

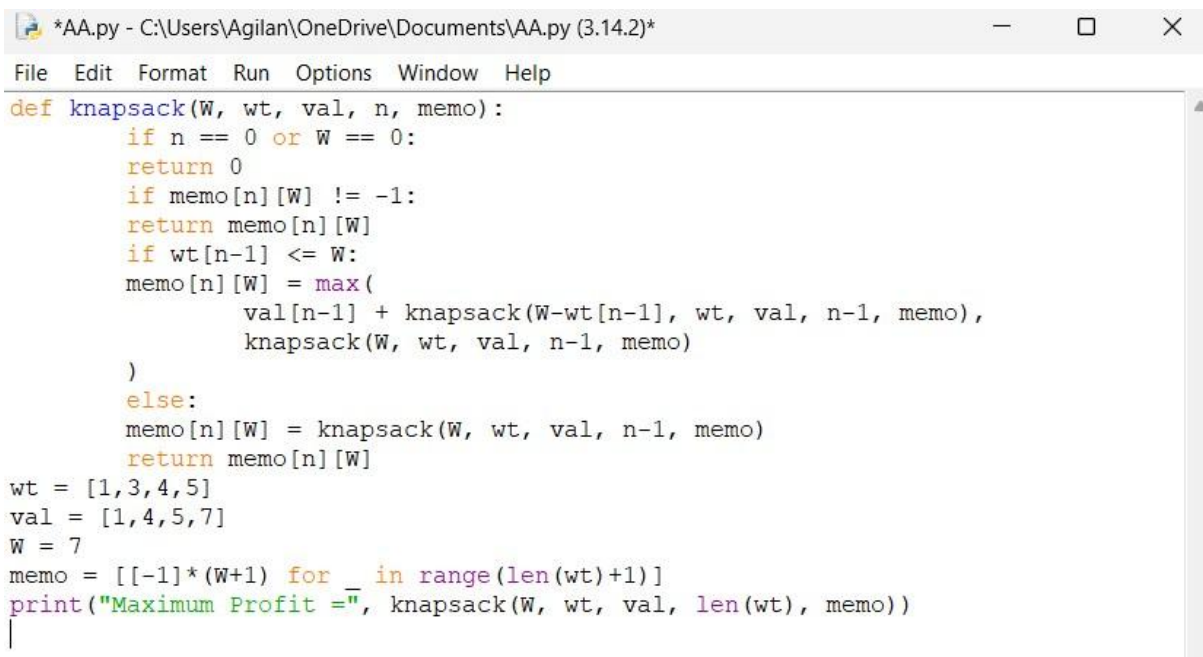
```

```

        return 0            if
memo[n][W] != -1:
return memo[n][W]    if
wt[n-1] <= W:
memo[n][W] = max(
    val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
    knapsack(W, wt, val, n-1, memo)
)
else:
    memo[n][W] = knapsack(W, wt, val, n-1, memo)
return memo[n][W]
wt = [1,3,4,5] val = [1,4,5,7] W = 7 memo = [[-1]*(W+1) for _ in
range(len(wt)+1)] print("Maximum Profit =", knapsack(W, wt,
val, len(wt), memo))

```

EXECUTION & OUTPUT:



```

*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]
    if wt[n-1] <= W:
        memo[n][W] = max(
            val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
            knapsack(W, wt, val, n-1, memo)
        )
    else:
        memo[n][W] = knapsack(W, wt, val, n-1, memo)
    return memo[n][W]
wt = [1,3,4,5]
val = [1,4,5,7]
W = 7
memo = [[-1]*(W+1) for _ in range(len(wt)+1)]
print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))

```

SUBSET SUM PROBLEM

PYTHON CODE:

```

def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in
range(n+1)]    for i in range(n+1):
dp[i][0] = True    for i in range(1, n+1):
for j in range(1, target+1):    if arr[i-1] <= j:

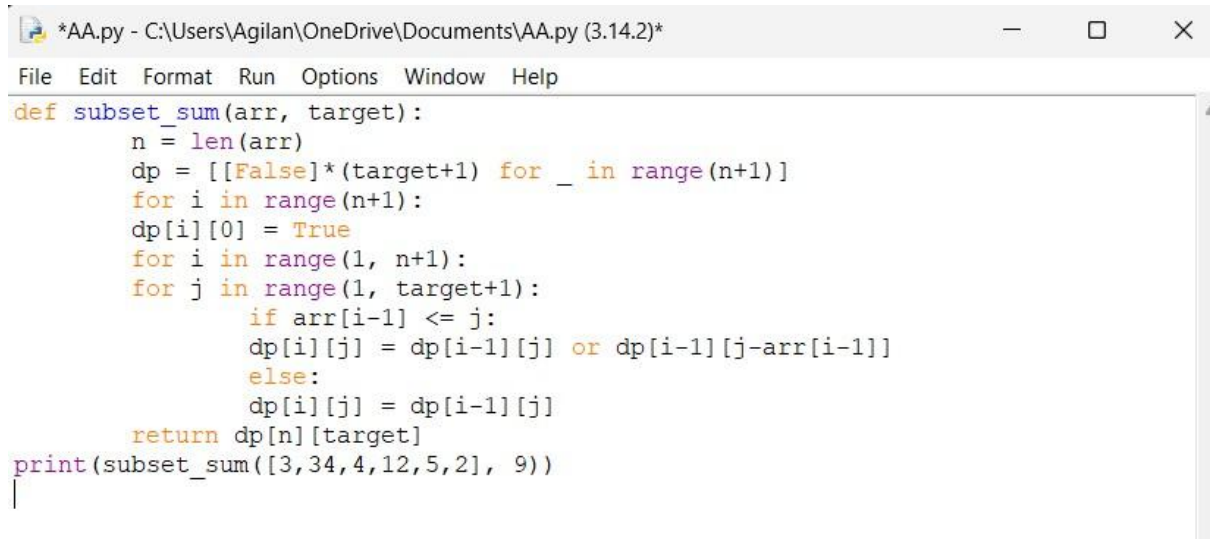
```

```

        dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
else:
    dp[i][j] = dp[i-1][j]
return dp[n][target]
print(subset_sum([3,34,4,12,5,2], 9))

```

EXECUTION& OUTPUT:



The screenshot shows a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The code in the editor is as follows:

```

def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]
    for i in range(n+1):
        dp[i][0] = True
    for i in range(1, n+1):
        for j in range(1, target+1):
            if arr[i-1] <= j:
                dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
            else:
                dp[i][j] = dp[i-1][j]
    return dp[n][target]
print(subset_sum([3,34,4,12,5,2], 9))

```

TRAVELLING SALESPERSON PROBLEM (DP VERSION)

PYTHON CODE:

```

from functools import lru_cache
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
n = len(graph)
@lru_cache(None)
def tsp(mask, pos):
    if mask == (1<<n)-1:
        return graph[pos][0]
    ans = float('inf')
    for city in range(n):
        if not mask & (1<<city):
            ans = min(ans, graph[pos][city] + tsp(mask | (1<<city), city))

```

```

        return ans
print("Minimum Cost =", tsp(1,0))

```

EXECUTION& OUTPUT:



```

*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
from functools import lru_cache
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
n = len(graph)
@lru_cache(None)
def tsp(mask, pos):
    if mask == (1<<n)-1:
        return graph[pos][0]
    ans = float('inf')
    for city in range(n):
        if not mask & (1<<city):
            ans = min(ans,
                graph[pos][city] +
                tsp(mask | (1<<city), city))
    return ans
print("Minimum Cost =", tsp(1,0))

```

TRAVELLING SALESPERSON USING DP TABLE

PYTHON CODE:

```

def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')]*n for _ in range(1<<n)]
    dp[1][0] = 0
    for mask in range(1<<n):
        for u in range(n):
            if mask & (1<<u):
                for v in range(n):
                    if not mask & (1<<v):
                        dp[mask | (1<<v)][v] = min(
                            dp[mask | (1<<v)][v],
                            dp[mask][u] + graph[u][v]
                        )
    ans = float('inf')
    for i in range(1,n):
        ans = min(ans, dp[(1<<n)-1][i] + graph[i][0])
    return ans
graph = [

```

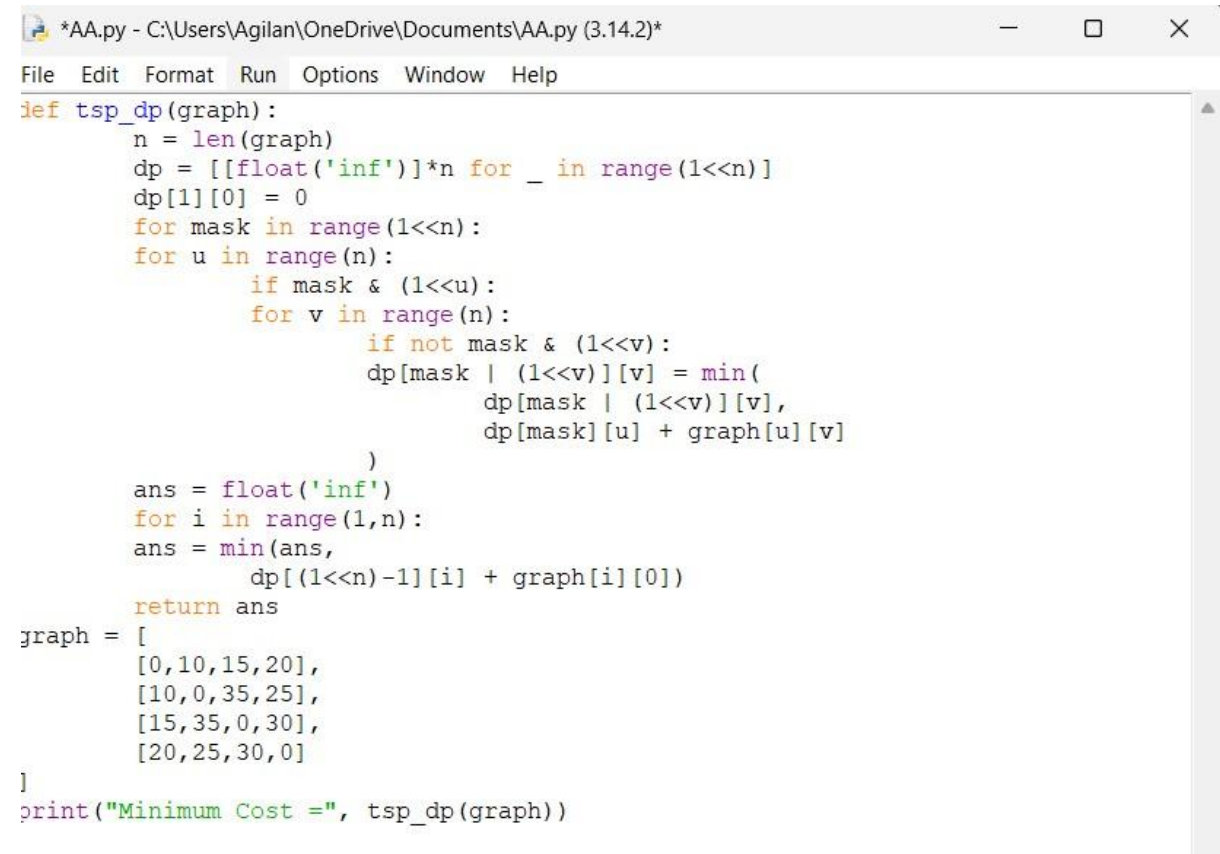


```

        [0,10,15,20],
        [10,0,35,25],
        [15,35,0,30],
        [20,25,30,0]
    ]
    print("Minimum Cost =", tsp_dp(graph))

```

EXECUTION& OUTPUT:



```

*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')] * n for _ in range(1 << n)]
    dp[1][0] = 0
    for mask in range(1 << n):
        for u in range(n):
            if mask & (1 << u):
                for v in range(n):
                    if not mask & (1 << v):
                        dp[mask | (1 << v)][v] = min(
                            dp[mask | (1 << v)][v],
                            dp[mask][u] + graph[u][v]
                        )
    ans = float('inf')
    for i in range(1, n):
        ans = min(ans,
            dp[(1 << n) - 1][i] + graph[i][0])
    return ans
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
print("Minimum Cost =", tsp_dp(graph))

```

OPTIMAL BINARY SEARCH TREE

PYTHON CODE:

```

def optimal_bst(freq):
    n = len(freq)
    cost = [[0] * n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i+L-1
            cost[i][j] =
            float('inf')
            total =

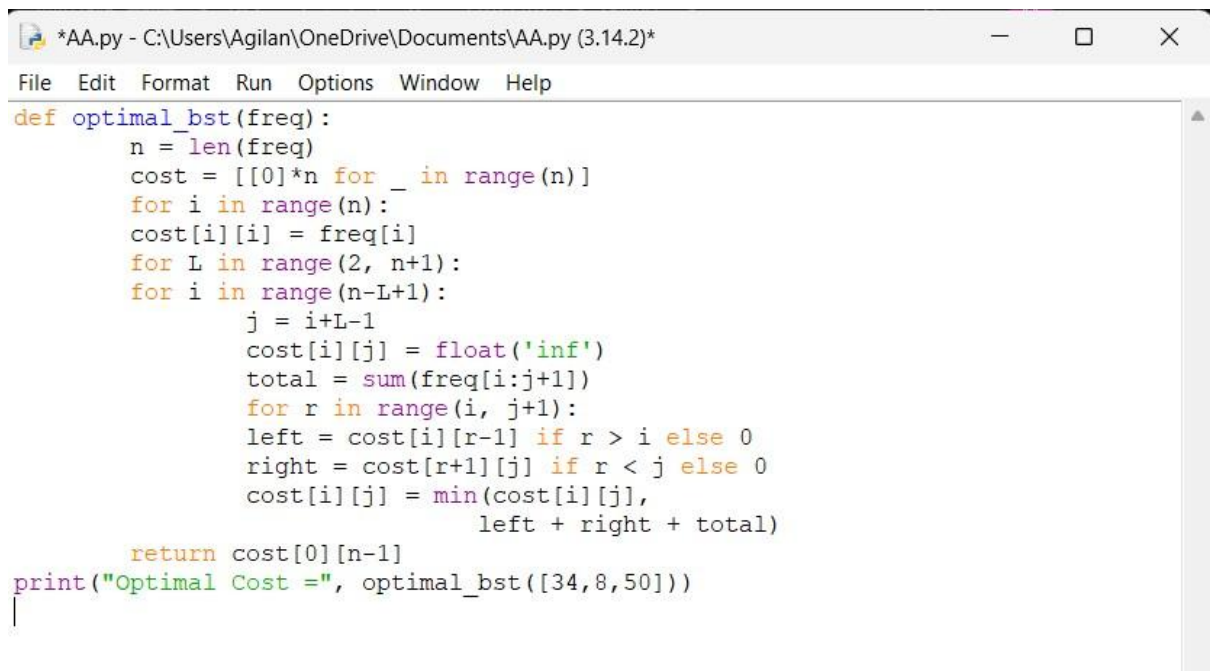
```

```

sum(freq[i:j+1])      for r in
range(i, j+1):
    left = cost[i][r-1] if r > i else 0
right = cost[r+1][j] if r < j else 0
cost[i][j] = min(cost[i][j],
    left + right + total)
return cost[0][n-1]
print("Optimal Cost =", optimal_bst([34,8,50]))

```

EXECUTION& OUTPUT:



The screenshot shows a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The code in the editor is as follows:

```

def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i+L-1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                cost[i][j] = min(cost[i][j],
                    left + right + total)
    return cost[0][n-1]
print("Optimal Cost =", optimal_bst([34,8,50]))

```

MATRIX CHAIN MULTIPLICATION

PYTHON CODE:

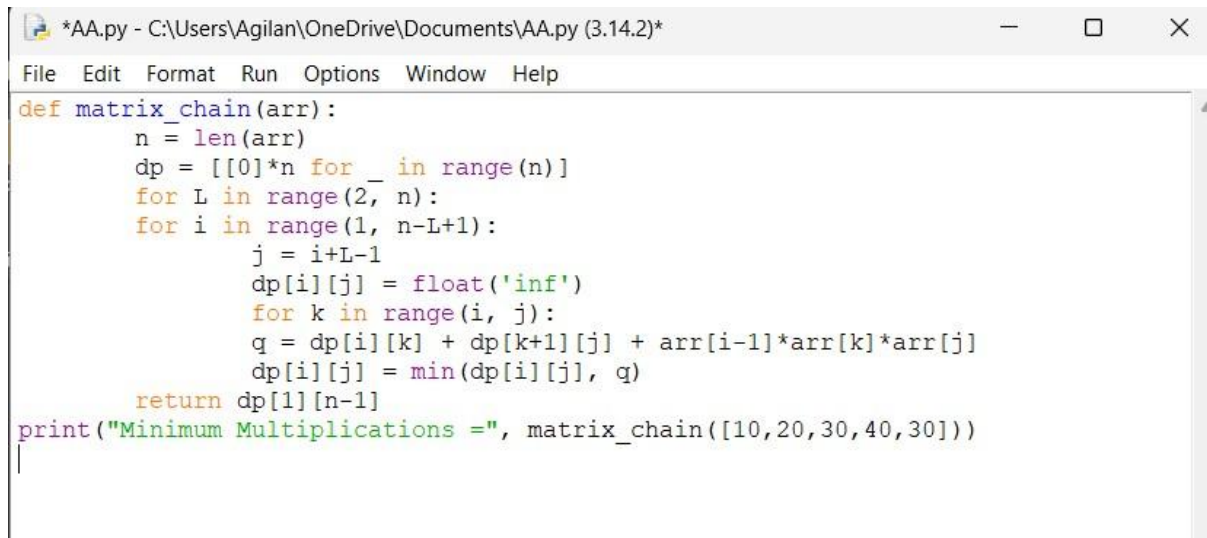
```

def matrix_chain(arr):
    n = len(arr)
    dp = [[0]*n for _ in
range(n)]      for L in range(2, n):
for i in range(1, n-L+1):
    j = i+L-1
    dp[i][j] = float('inf')
    for k in range(i, j):
        q = dp[i][k] + dp[k+1][j] + arr[i-
1]*arr[k]*arr[j]
        dp[i][j] = min(dp[i][j], q)
    return dp[1][n-1]

```

```
print("Minimum Multiplications =", matrix_chain([10,20,30,40,30]))
```

EXECUTION& OUTPUT:

A screenshot of a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The window contains the following code:

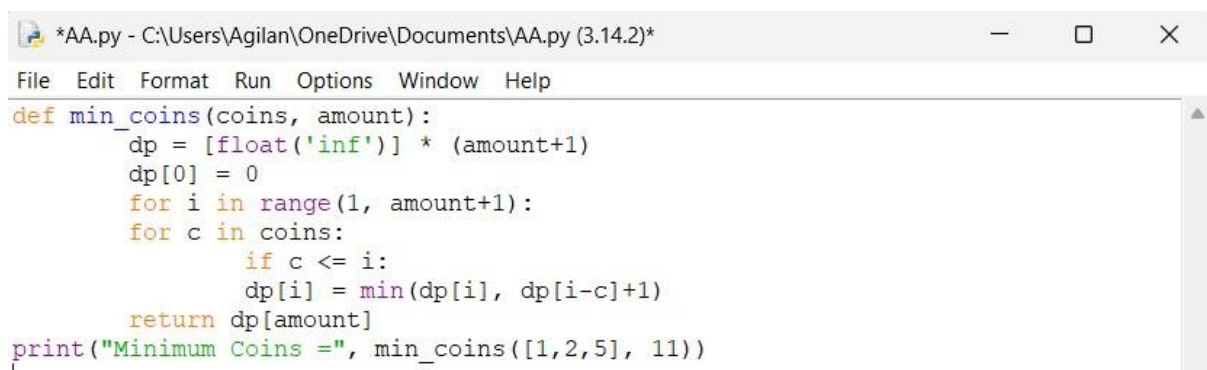
```
def matrix_chain(arr):  
    n = len(arr)  
    dp = [[0]*n for _ in range(n)]  
    for L in range(2, n):  
        for i in range(1, n-L+1):  
            j = i+L-1  
            dp[i][j] = float('inf')  
            for k in range(i, j):  
                q = dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j]  
                dp[i][j] = min(dp[i][j], q)  
    return dp[1][n-1]  
print("Minimum Multiplications =", matrix_chain([10,20,30,40,30]))
```

COIN CHANGE (Minimum Coins)

PYTHON CODE:

```
def min_coins(coins, amount):  
    dp = [float('inf')] * (amount+1)  
    dp[0] = 0  
    for i in range(1, amount+1):  
        for c in coins:  
            if c <= i:  
                dp[i] = min(dp[i], dp[i-c]+1)  
    return dp[amount]  
print("Minimum Coins =", min_coins([1,2,5], 11))
```

EXECUTION& OUTPUT:

A screenshot of a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The window contains the following code:

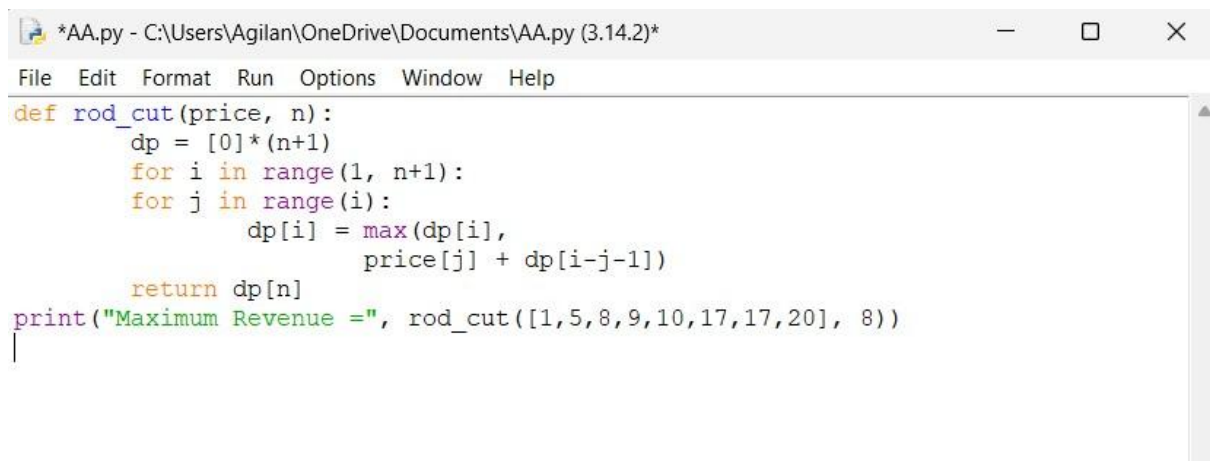
```
def min_coins(coins, amount):  
    dp = [float('inf')] * (amount+1)  
    dp[0] = 0  
    for i in range(1, amount+1):  
        for c in coins:  
            if c <= i:  
                dp[i] = min(dp[i], dp[i-c]+1)  
    return dp[amount]  
print("Minimum Coins =", min_coins([1,2,5], 11))
```

ROD CUTTING PROBLEM

PYTHON CODE:

```
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i],
                price[j] + dp[i-j-1])
    return dp[n]
print("Maximum Revenue =", rod_cut([1,5,8,9,10,17,17,20], 8))
```

EXECUTION& OUTPUT:

A screenshot of a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code editor displays the same Python code as the previous block. The output of the program is visible at the bottom of the window, showing 'Maximum Revenue =' followed by a blank line, indicating the result of the calculation.

```
*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*
File Edit Format Run Options Window Help
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i],
                price[j] + dp[i-j-1])
    return dp[n]
print("Maximum Revenue =", rod_cut([1,5,8,9,10,17,17,20], 8))
|
```

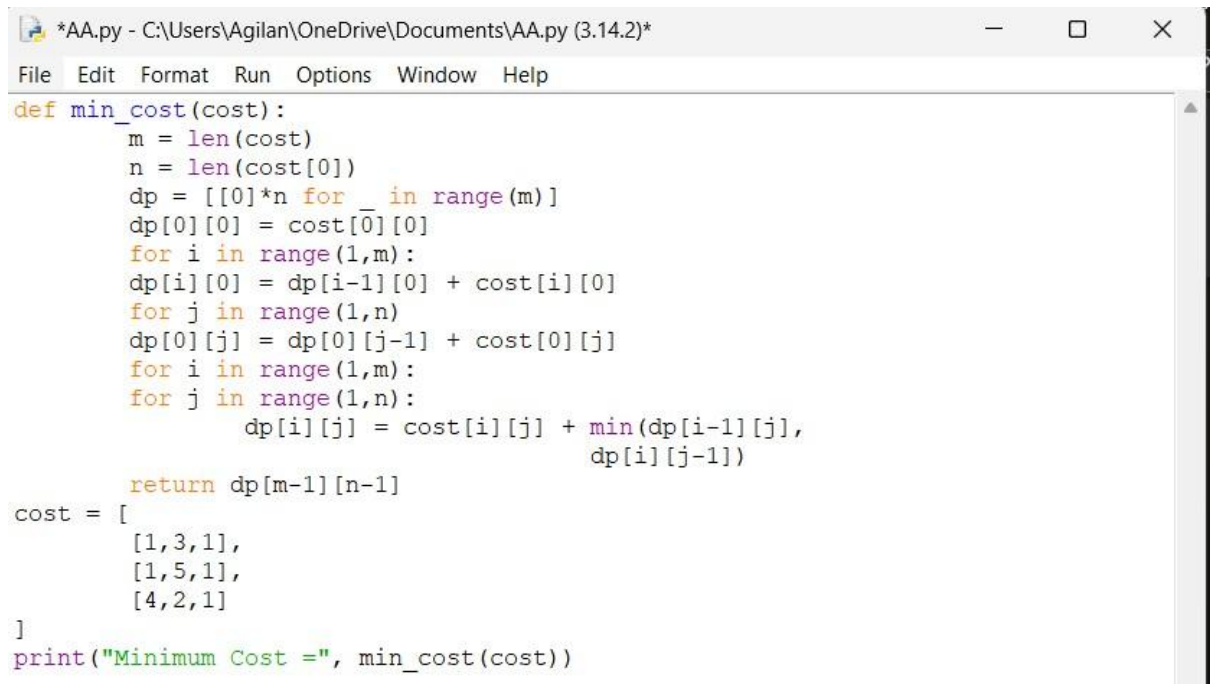
MINIMUM COST PATH

PYTHON CODE:

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1,m):
        dp[i][0] = dp[i-1][0] + cost[i][0]
    for j in range(1,n):
        dp[0][j] = dp[0][j-1] + cost[0][j]
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = cost[i][j] + min(dp[i-1][j],
                dp[i][j-1])
    return dp[m-1][n-1]
```

```
cost = [  
    [1,3,1],  
    [1,5,1],  
    [4,2,1]  
]  
print("Minimum Cost =", min_cost(cost))
```

EXECUTION& OUTPUT:



The screenshot shows a Python IDE window titled '*AA.py - C:\Users\Agilan\OneDrive\Documents\AA.py (3.14.2)*'. The window contains the following code:

```
def min_cost(cost):  
    m = len(cost)  
    n = len(cost[0])  
    dp = [[0]*n for _ in range(m)]  
    dp[0][0] = cost[0][0]  
    for i in range(1,m):  
        dp[i][0] = dp[i-1][0] + cost[i][0]  
    for j in range(1,n):  
        dp[0][j] = dp[0][j-1] + cost[0][j]  
    for i in range(1,m):  
        for j in range(1,n):  
            dp[i][j] = cost[i][j] + min(dp[i-1][j],  
                                         dp[i][j-1])  
    return dp[m-1][n-1]  
  
cost = [  
    [1,3,1],  
    [1,5,1],  
    [4,2,1]  
]  
print("Minimum Cost =", min_cost(cost))
```